

Dynamic Adjustment of Monte Carlo Tree Search

Simulations for a 9×9 -Go-AI

Contents

1	Introduction	3
1.1	Gap	3
1.2	Hypothesis	3
2	Background knowledge	3
2.1	Go	3
3	Technical Foundations	4
3.1	Reinforcement Learning and Self-Play FILL ML nec?	4
3.2	Deep Neural Networks	5
3.2.1	Artificial Neural Networks	5
3.2.2	Residual Neural Networks	5
3.3	Monte Carlo Tree Search	6
3.4	BayesElo	6
4	Previous work	8
4.1	Pre-deep Learning	8
4.1.1	Before MCTS	8
4.1.2	After MCTS	8
4.2	Deep Learning	9
4.2.1	AlphaGo	9
4.2.2	AlphaZero	10
4.2.2.1	Architectural Framework: The Dual-Head ResNet	10
4.2.2.1.1	The Policy Head (p)	10
4.2.2.1.2	The Value Head (v)	10
4.2.2.2	Search Mechanism: Guided MCTS	10
4.2.2.2.1	Selection and the PUCT Algorithm	10
4.2.2.3	The Reinforcement Learning Pipeline	11
4.2.3	KataGo	11
4.2.4	Impact on the Go Community	12
4.2.5	Further research	12
5	Methodology	12
5.1	Reasoning 9×9 vs. 19×19	12
5.1.1	Complexity and State-Space Reduction	13
5.1.2	Tactical Density and Heuristic Sensitivity	13
5.1.3	Statistical Significance through Volume	13

5.2 Reasoning AlphaZero vs. KataGo	13
5.3 Environment (Training server, implementation)	14
5.3.1 Computational Resources: University HPC Cluster	15
5.3.1.1 Hardware Configuration	15
5.3.1.2 Job Management and Scalability	15
5.4 Heuristics	15
5.4.1 Difference of Probability for Likeliest K Actions	15
5.4.1.1 The Mathematical Logic	15
5.4.1.2 Reasoning: Strategic Bifurcation	16
5.4.1.3 Budget Scaling Function	16
5.4.2 Difference of Highest and Lowest K Actions	16
5.4.2.1 The Mathematical Logic	16
5.4.2.2 Reasoning: Constrained vs. Unconstrained States	17
5.4.3 Learning Complexity	17
5.4.3.1 The Adaptive Simulation Heuristic	17
5.4.3.2 Training the Regression Head	17
5.4.3.3 Heuristics Leveraging NN Output	17
5.4.4 Filtering Heuristic	18
5.4.5 Random	18
5.4.5.1 The “Fast vs. Slow” Search Heuristic	19
5.4.5.2 Reasoning for Randomization in 9x9 Go	19
5.5 Evaluation	19
5.5.1 Self-Play Tournaments and Round-Robin Analysis	19
5.5.2 Strength Ranking: The BayesElo System	20
5.5.3 Efficiency Metrics: Elo-per-Simulation	20
6 Experiments & Results	20
6.1 Using Heuristics	20
6.2 Using Heuristics in Play	20
7 Discussion	20
8 Conclusion	20
Bibliography	21

1 Introduction

Creating a computer capable of playing games has been part of Computer Science research for a long time. It is a benchmark of human intelligence and an exploration of the boundaries of what is capable using artificial decision making. This culminated in the defeat of reigning world-chess champion, Garry Kasparov, at the computations of the IBM chess computer DeepBlue in 1997. [CITE HERE](#) This spurred on further improvement on chess machines and research on a multitude of other games. Another popular game by the name of Go remained a problem to scientists for 19 more years. While many approaches were tested, it was not until 2016 that a high-ranking professional player was beaten by Google-DeepMind's artificial intelligence (AI), AlphaGo, that used a vastly different approach from earlier game computers and programs. It combined Deep Neural Network (DNN) training with exploration guided by distributions output by the neural network. [CITE HERE](#)

1.1 Gap

A problem that is apparent with most AIs is the amount of computation that is necessary to achieve good results. The literature is abundant with examples of large networks and decision making algorithms that reasonably require too much computing power. Therefore, it is necessary to improve efficiency of algorithms by all possible means. Especially, algorithms that make use Monte Carlo Tree Search (MCTS) seem to tend to execute a lot of unnecessary computation because it executes the allowed number of simulations without consideration as to whether the return move probability distribution changes and such computation is necessary.

1.2 Hypothesis

The central claim of this thesis is that efficiency of MCTS can be increased by dynamically adjusting the amount of simulations based on the prior move probabilities generated by the DNN while upholding parity in performance. The development and implementation of such a heuristic are the goal of this thesis.

2 Background knowledge

2.1 Go

Go is a Chinese board game that can be traced back to 4,000 years ago. It is played by two-players using black and white stones. While usually played on a 19×19 -grid, other board sizes like 9×9 or 13×13 exist. The goal of the game is to enclose more territory, i.e., grid intersections, than the opponent.

The players place stones in turn; the player with the black stones begins. The group of free places around a single stone are called liberties, see Figure 1. Stones of the same colour that are placed on neighbouring fields are considered a group and share the liberties. A

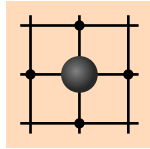


Figure 1: Liberties of stones

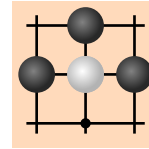


Figure 2: Capturing a Stone

group may be arbitrarily large. A group of the opponent can be captured by filling in all liberties of the group, see Figure 2. While generally any stone may be placed anywhere on the board, it may not be placed in places where the stone or its group dies because of the stone being placed. However, such a stone may be placed if placing this stone means capturing an opponent's group. Importantly, a board position may not be immediately repeated. This means, after a stone has been captured, and the capturing stone may not be recaptured on the next move. This is called Ko. **FILL Appendix**

The game ends when both players pass consecutively. Usually, players pass when all necessary moves are played and the territories are firmly established. Afterwards, the territory is counted. For that, players place back their prisoner stones into the opponent's territory. The remaining free grid intersections within their territory are the points for the respective player. To account for black's advantage in going first and avoid draw, additional 5.5-7.5 points are awarded to white. This is called komi.

Komi can be adjusted to even out the differences in capability between differently strong players. Lastly, komi can be combined with a handicap that places black stones on predetermined spots on the board.

Traditionally, players, human or computational, are attributed ranks based on strength. The weakest players play at 30 kyu. You can improve until reaching 1 kyu. From there, you can improve to 1 dan and theoretically up to 9 dan. Most top-level amateur players become 6 dan. Professional players are ranked separately with their own dan-scale. Nowadays, players are also evaluated using the Elo-system.

3 Technical Foundations

3.1 Reinforcement Learning and Self-Play **FILL ML nec?**

Reinforcement Learning (RL) describes a Machine Learning paradigm in which an agent is trained to maximize a reward signal in interaction with its environment. This makes use of two concepts called exploitation and exploration. To exploit is to choose a path of action that is known to produce a good result. On the other side, to explore is to choose a path of action that is not well-known and thus, upon further examination, may produce a better result than already known paths of action. Self-play is a specialised framework within the field of RL. In self-play, an algorithm plays a match in a game against an earlier iteration of itself. In this case, the previous iteration is the environment and the reward signal is the

outcome of the match. Various selection strategies exist for the comparison-iteration, e.g., a random iteration is chosen to play, or the selection follows a “King-of-the-Hill”-approach, where the strongest iteration remains as the benchmark and is to be beaten. This, more than the randomised selection shows growth in performance of the agent.

3.2 Deep Neural Networks

3.2.1 Artificial Neural Networks

Today, artificial neural networks (ANN) are one of the most used and researched technologies. With their groundwork laid in 1958, new methods have been since been developed for a large number of purposes, not the least of which is playing games and improving beyond human capabilities.

Generally, ANNs receive an input of defined but arbitrary format. The signal of this input are propagated through the network to the output layer. One layer is made up of multiple neurons, each with their own weights per input, plus a bias individual to each neuron. In the context to RL, the ANN’s output is evaluated for its reward signal in response to the environment. From this, an error signal is generated and passed backwards through the network. Each layer is updated dependent on the succeeding layer’s error signal.

3.2.2 Residual Neural Networks

In the course of research and development of ANNs, they have grown exponentially deeper. Beginning with a single perceptron, which were then used in a single layers, now, there are networks using neurons with more than 100 layers and a large number of perceptrons per layer. Users of ANNs of this size noticed the “Vanishing Gradient” problem. In the process of backpropagation, the derivatives’ chain rule is employed. For that, multiple small decimal numbers are repeatedly multiplied. This produces even smaller decimal numbers The “Vanishing Gradient” describes the phenomenon that the error signal that early layers of the networks receive is too small to effectively change the weights and biases of those layers. This stops learning from having any effect.

Hello

Figure 3: Illustration of Residual Neural Network

To solve this problem, residual neural networks were introduced in Deep Learning (DL) problems. Residual neural networks (ResNet) make use of “skipping connections.” In one of such connections, a residual layer projects its output to another layer further down the stack of layers, skipping the layers inbetween. The residual output is added to the output of the layer, to which it is is projected. This largely fixes the Vanishing Gradient since the output of the layer is passed further into the stack of layers.

A negative point in this setup is that the intermediate layers may be found to be superfluous and converge to 0 during training.

3.3 Monte Carlo Tree Search

MCTS is a heuristic decision making algorithm. Through exploration of subsequent states from a root state, MCTS finds a state that is most likely to maximise a metric that can be changed per use-case. During the search, a tree is built from the original state as root node R. There are four stages to MCTS:

1. Selection: From a parent node P, choose a child node according to a selection method. Repeat until choosing a leaf node C.
2. Expansion: If C is not a terminal state, i.e., there is a possible following state, create new child nodes. Choose a child state.
3. Evaluation: A state may be evaluated by choosing random subsequent states until a terminal state which has a definitive value. Another method is an evaluation function applied to the state.
4. Backpropagation: Update the probabilities in the path from C to R using the result from the rollout.

Through rigorous exploration of the tree by simulations, i.e., paths from root to terminal state, the child states most likely to maximise the evaluation method converge to higher values in the selection method.

The selection method most commonly used in the selection phase is the “Upper Confidence Bound 1 applied to Trees” (UCT). It is used to find a balance between exploitation of well-simulated moves and exploration of less-simulated moves in MCTS. In the selection stage, the node with the highest value according to the expression

$\frac{w_i}{n_i} + c\sqrt{\ln \frac{N_i}{n_i}}$ is selected, where

- w_i is the the wins for prospective C after the i -th move,
- n_i is the visits for prospective C after the i -th node,
- N_i is the visits for P after the i -th move, and
- c is the exploration parameter.

3.4 BayesElo

While simple win rates provide a snapshot of performance, they are sensitive to the specific pairings of a tournament. The BayesElo algorithm is a Bayesian refinement of the classic Elo rating system, specifically designed to handle the “Komi” (first-move advantage) and the possibility of draws—though draws are virtually non-existent in Go due to the fractional 7.5 point Komi. The Mathematical Framework

BayesElo assumes that the performance of a model i in a given game is a random variable following a logistic distribution centered at its true rating R_i . The probability P of model A defeating model B is expressed as:

$$P(A > B) = 1 + 10^{\frac{R_B - R_A}{400}}$$

In this system, a rating difference of 200 points signifies that the stronger model has a 75% expected win rate, while a 400-point lead indicates a 90% win rate.

4 Previous work

4.1 Pre-deep Learning

4.1.1 Before MCTS

Early Go programs were usually rule-based like Deep Blue or used pattern recognition to find hard-coded patterns and choose actions based on those patterns.

Albert Zobrist wrote an early Go program which was based on pattern recognition and was capable of calculating territory and playing Ko's through Zobrist hashing which still has uses to this day.

Another such program is "The Many Faces of Go" that was using a large compilation of rules and an influence function. It was vastly successful in Computer Go championships in 1998-2002. It was awarded an 8-kyu rank for winning a tournament in . It has since been continuously improved and made to embrace new technologies until the introduction of Deep Learning.

An open-source program that also works based on rules and pattern recognition is GnuGo, whose pipeline is publicly available. Within the pipeline, GnuGo first analyses the board for its state, examines groups, their life-and-death, and their relation to one another. Next, it generates the moves, and lastly evaluates the moves for its effect on influence and territory. The move that has the best evaluation is played.

These models were found to perform well in specific parts of the game like Life-and-Death-Problems or **FILL Something Else**. However they were unable to play a complete game well or even better than amateur players. Programs of this level of sophistication could not achieve the same level of success for Go as for chess, because the computational search for Go is drastically different. While it is not the only factor, the search space is much larger on a large board, $3^{19 \times 19} \approx 10^{170}$. However, Go on a 9×9 -board has a similar branching factor to chess and has proved equally as difficult as Go on a large board. This is due static position evaluation requiring a large amount of local auxiliary computations. Additionally, in order to find legal moves, the history of the game has to be contained within the state information to account for special rules such as Ko.

4.1.2 After MCTS

With the introduction of MCTS into the field of Computer Go, programs became drastically better.

The "Many Faces of Go"-program improved to 1 dan.

Other models such as **FILL** managed to play amateurs and professionals with less handicap than before.

4.2 Deep Learning

4.2.1 AlphaGo

FILL Also shorten AlphaGo because that is not the important algorithm

FILL Replace greek letters with clearer naming The AlphaGo-algorithm is a complex system of neural networks, MCTS, supervised learning (SL), and reinforcement learning (RL). SL policy network p_σ and SL sample network p_π were trained from a database of expert-level moves. p_π was trained to accurately predict human moves. p_π was designed to quickly sample actions during the rollout in the simulation of MCTS. RL policy network p_ρ on the other hand is trained to optimize the outcome of self-play. The RL value network $v^{p(s)}$ predicts the outcome using policy p .

Supervised learning. The first policy network $p_{\pi(a|s)}$ receives as input a board representation s and gives out move j a probability distribution over all legal moves a . It optimizes the selection of action a , given s , using stochastic gradient descent according to $\Delta\sigma \propto \frac{\delta \log p_\sigma(a|s)}{\sigma}$.

Reinforcement learning. The second policy network $p_{\pi(a|s)}$ is architecturally the same, with the same initial weights $\rho = \pi$ and trained with policy gradient reinforcement learning. It also outputs a probability distribution over all legal moves a . Self-play is done with a current and a random previous iteration of the network. The reward function is zero for all non-terminal time steps and either +1 or -1 for winning and losing, respectively. The loss is thus $\Delta\rho \propto \frac{\delta \log p_\rho(a_t|s_t)}{\rho} z_t$.

The value function $v^{p(s)}$ approximates the perfect play $p^*(s)$ using the best available policy p from p_ρ to $v^{p(s)} = \mathbb{E}[z_t | s_t = s, a_{t..T} \sim p]$. The value network $v_{\theta(s)}$ approximates that function. Architecturally, v_θ is similar to the policy networks but is trained on state-outcome pairs (s, z) using stochastic gradient descent to minimize the mean-squared error (MSE) between predicted value $v_{\theta(s)}$ and the actual outcome z $\Delta\theta \propto \frac{\delta v_{\theta(s)}}{\delta\theta} (z - v_{\theta(s)})$.

Search In the MCTS, each edge (s, a) holds record of an action value $Q(s, a)$, visit count $N(s, a)$, and prior probability $P(s, a)$. The tree is traversed by selection of a child at time step t by

$$a_t = \underset{a}{\operatorname{argmax}} (Q(s_t, a) + u(s_t, a))$$

with a bonus

$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)}$$

which encourages exploration by disfavouring much-visited edges.

A leaf node L is processed once by p_π which outputs the prior probabilities $P(a|s) = p_\pi(a|s)$. $v_{\theta(s_L)}$ and $p_{\sigma(s_L)}$ **CHECK** are used to generate an evaluation $V(s_L)$ of the state of the leaf node weighted by a mixing factor λ :

$$V(s_L) = (1 - \lambda)v_{\theta(s_L)} + \lambda z_L$$

At the end of one iteration of search, visited edges are updated. Visits through all search iterations are counted and mean evaluation calculated to

$$N(s, a) = \sum_{i=1}^n 1(s, a, i)$$

$$N(s, a) = \sum_{i=1}^n 1(s, a, i)$$

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^n 1(s, a, i) V(s_L^i)$$

where s_L^i is the leaf node L in the i th simulation. $1(s, a, i)$ shows whether that node was visited in the i th iteration.

4.2.2 AlphaZero

4.2.2.1 Architectural Framework: The Dual-Head ResNet

The core of AlphaZero is a deep convolutional neural network f_θ with parameters θ . Unlike traditional engines that utilize separate evaluation and policy functions, AlphaZero employs a unified “two-headed” architecture.

Given a board state s , the network produces two primary outputs:

$$f_{\theta(s)} = (p, v)$$

4.2.2.1.1 The Policy Head (p)

The policy head outputs a vector p representing a probability distribution over the action space a . Formally, $p_a = P(a | s)$. This serves as a “prior” to guide the search toward the most promising branches of the game tree.

4.2.2.1.2 The Value Head (v)

The value head outputs a scalar $v \in [-1, 1]$, estimating the expected outcome z of the game from the current state s . This learned value function replaces the traditional “rollout” phase of MCTS, significantly reducing the variance of the state evaluation.

4.2.2.2 Search Mechanism: Guided MCTS

AlphaZero utilizes an asynchronous policy and value MCTS. Each node in the search tree stores statistics: the visit count $N(s, a)$, the total action value $W(s, a)$, and the mean action value $Q(s, a) = \frac{W(s, a)}{N(s, a)}$.

4.2.2.2.1 Selection and the PUCT Algorithm

During the selection phase, the algorithm traverses the tree by selecting actions that maximize the Polynomial Upper Confidence Trees (PUCT) formula:

$$a_t = \operatorname{argmax}_a (Q(s, a) + U(s, a))$$

The exploration term $U(s, a)$ is defined as:

$$U(s, a) = C_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

This mechanism ensures a rigorous balance between exploitation (choosing moves with high Q values) and exploration (visiting moves with high prior probability P but low visit counts N).

4.2.2.3 The Reinforcement Learning Pipeline

The learning process is an iterative cycle of self-play and optimization.

1. **Self-Play:** The agent executes MCTS guided by the current network θ to play games against itself.
2. **Search MCTS:** For each state, the search produces a search probability distribution π .
3. **Optimization:** The parameters θ are adjusted via stochastic gradient descent to minimize the loss function L :

$$L = (z - v)^2 - \pi^T \log \mathbf{p} + c \|\theta\|^2$$

Where:

- $(z - v)^2$ is the value error (Mean Squared Error).
- $\pi^T \log \mathbf{p}$ is the policy error (Cross-Entropy Loss).
- $c \|\theta\|^2$ is the L_2 weight decay for regularization.

4.2.3 KataGo

KataGo, developed by David J. Wu (lightvector), represents a significant evolution in the lineage of Go-playing artificial intelligence. While its predecessors, like DeepMind's AlphaZero, demonstrated that superhuman strength could be achieved through pure self-play reinforcement learning, KataGo introduced several architectural and algorithmic innovations that prioritized efficiency, flexibility, and utility for human analysis. Key Innovations and Architecture

Unlike the original AlphaZero, which required massive computational resources to train, KataGo was designed to learn more quickly and provide deeper insights into the game. Its primary contributions include:

- Auxiliary Training Tasks: Beyond predicting the move (policy) and the winner (value), KataGo's neural network is trained on additional "heads." These include predicting the final score, territory ownership, and the number of stones remaining. This extra information forces the network to learn the underlying nuances of the game state more effectively.
- Global Context (Global Pooling): KataGo utilizes global pooling layers that allow the network to "see" the entire board at once, facilitating a better understanding of interconnected groups and large-scale strategic trade-offs that standard convolutional layers might miss.

- **Dynamic Komi and Ruleset Flexibility:** Most AI models are trained on a fixed ruleset (typically Chinese rules with a 7.5 komi). KataGo is uniquely versatile, capable of adjusting its play for different komi values, board sizes, and even different rulesets (Japanese, Tromp-Taylor, etc.) without retraining.
- **Playout Cap Randomization:** By varying the number of “playouts” (simulations) during training, KataGo becomes proficient at both quick, intuitive play and deep, calculated reading, making it more robust against adversarial attacks and “blind spots.”

4.2.4 Impact on the Go Community

KataGo has become the gold standard for Go players and researchers alike. Because it provides a predicted score (e.g., “Black is leading by 4.5 points”) rather than just a win percentage, it offers a more “human” way to evaluate positions.

- **Technical Significance:** In many positions where the win rate is close to 100% or 0%, traditional bots stop providing useful feedback. KataGo continues to provide accurate score leads, making it indispensable for handicap games and endgame analysis.

4.2.5 Further research

Recent research has moved beyond simple parameter tuning to structural modifications of the MCTS process itself:

- **Monte Carlo Graph Search (MCGS):** A notable advancement where the search tree is generalized into a Directed Acyclic Graph (DAG). This allows the AI to recognize transpositions—different sequences of moves that lead to the identical board state—significantly reducing redundant computations on the 9x9 board.
- **Terminal Solvers:** Since 9x9 Go is a smaller domain, some implementations integrate “mini-max” style terminal solvers within the MCTS leaves. If a branch can be proven as a win or loss through exhaustive search, the result is backpropagated immediately, pruning the tree.
- **Warm-Start MCTS:** Researchers have experimented with using classical heuristics (like RAVE) to “warm-start” the MCTS during the early stages of AlphaZero self-play training, mitigating the high failure rate of agents starting with purely random weights.

5 Methodology

5.1 Reasoning 9×9 vs. 19×19

The selection of the 9×9 board for this research is driven by the necessity of high-frequency iteration and statistical saturation. While the 19×19 board remains the ultimate goal for Go AI, it introduces significant noise in the early stages of reinforcement learning that can obscure the effectiveness of specific search heuristics

5.1.1 Complexity and State-Space Reduction

The 19×19 board is widely considered computationally “unsolved” in the context of reinforcement learning due to its massive search depth and complexity. In contrast, the 9×9 board is small enough to permit:

- **Rapid Convergence:** The neural network parameters reach a “stable” state in hours rather than weeks, allowing for hundreds of experimental trials.
- **Optimal Policy Proximity:** On 9×9 , the gap between an AI’s policy $\pi_{\{\theta\}}$ and the theoretical optimal policy π^* is more easily measured via exhaustive solvers.

5.1.2 Tactical Density and Heuristic Sensitivity

The 9×9 format is highly “tactical.” Every stone played exerts immediate pressure on the opponent, forcing the model to learn life-and-death, connectivity, and sacrifice patterns much faster than on a 19×19 board, where the game is largely dominated by global influence and long-term strategic buildup. By utilizing the 9×9 board, we isolate the efficacy of our MCTS heuristics:

- **Search Efficiency:** We can observe if our “Forced Payouts” correctly identify tactical “kill” sequences that are often ignored in standard MCTS.
- **Policy Precision:** Because the state space is constrained, we can compare the predicted move probabilities against a brute-force search baseline to determine if our heuristics introduce bias or improve accuracy.

5.1.3 Statistical Significance through Volume

The primary advantage of the 9×9 board is the ability to generate orders of magnitude more self-play games. In a thesis context, this is crucial for establishing the statistical significance of your results. If a heuristic provides a 2% improvement in Elo over 10^5 games on a 9×9 board, the confidence intervals are significantly tighter than they would be on the 19×19 scale, where similar testing would be computationally prohibitive for a single researcher.

5.2 Reasoning AlphaZero vs. KataGo

While KataGo represents the current state-of-the-art in Go-playing AI, this research utilizes a core AlphaZero architecture as the experimental baseline. This choice is predicated on three primary factors: algorithmic isolation, codebase transparency, and the reduction of confounding variables.

3.5.1. Algorithmic Isolation and Ablation Integrity

KataGo is not a singular algorithm but rather a “bundle” of highly optimized features, including auxiliary ownership heads, score-prediction heads, global context pooling, and hard-coded ladder solvers.

The Confounding Variable Problem: If a new heuristic is tested on top of KataGo, it is difficult to determine whether the resulting performance gain is due to the heuristic itself or its interaction with one of KataGo’s many existing optimizations.

The “Vanilla” Advantage: AlphaZero provides a “clean room” environment. By using the vanilla AlphaZero framework, we can ensure that any observed improvements in search efficiency or win rate are directly attributable to the heuristics introduced in this thesis (e.g., Regret-Guided Search Control or Forced Payouts).

3.5.2. Codebase Transparency and Technical Debt

The KataGo codebase is highly optimized for performance (using C++, OpenCL/CUDA, and sophisticated memory management), which introduces significant technical debt for a research project.

Modifiability: Implementing new MCTS heuristics in KataGo requires navigating a complex, multi-threaded engine designed for speed rather than readability.

Implementation Simplicity: AlphaZero’s logic is fundamentally simpler. Utilizing a Python/PyTorch-based AlphaZero reimplementation allows for rapid prototyping of the tree-search logic. This ensures that the focus of the work remains on algorithmic innovation rather than low-level systems engineering.

CHECK While KataGo is a stronger program than AlphaZero, the domain-dependent optimizations of the model rely heavily on a thorough exploration of the search tree. On one hand, as the experiments of this thesis are intended to limit exactly that exploration, combining these methods may work to a negative effect. On the other hand, allowing the algorithm to stay domain-independent may allow the optimizations to be used more widely.

5.3 Environment (Training server, implementation)

The primary research tool for this study is OpenSpiel, an open-source framework developed by Google DeepMind for reinforcement learning in games. OpenSpiel was selected for its robust, community-validated implementation of the AlphaZero algorithm and its native support for the game of Go.

- **AlphaZero Implementation:** The project utilizes the `alpha_zero` algorithm within OpenSpiel, which provides a unified architecture for Monte Carlo Tree Search (MCTS) and deep neural networks.
- **Language & Libraries:** While the core game logic of 9x9 Go is implemented in C++ for high-performance state transitions, the training loop and heuristic modifications are managed via the Python API. Neural network operations are handled using TensorFlow/LibTorch (specify which one you used), leveraging specialized kernels for GPU acceleration.
- **Game Representation:** The 9x9 Go environment in OpenSpiel follows standard Tromp-P Taylor rules. The state is represented as a $9 \times 9 \times 17$ tensor, encoding the current board position, previous moves (for history/ko), and the current player’s color.

5.3.1 Computational Resources: University HPC Cluster

Due to the high computational cost of self-play reinforcement learning, all training and evaluation experiments were conducted on the University's High-Performance Computing (HPC) Cluster.

5.3.1.1 Hardware Configuration

The cluster provides a distributed environment optimized for deep learning. The specific resources allocated for this project include:

- Compute Nodes: Training was distributed across [X] nodes, each equipped with [Y] CPU cores (e.g., AMD EPYC or Intel Xeon) to handle the parallel MCTS simulations.
- Accelerators: Neural network training and inference during self-play were accelerated using NVIDIA [Model, e.g., A100/V100] GPUs, providing the necessary TFLOPS for high-throughput batch processing.
- Memory: Each node provided [Z] GB of RAM to maintain large MCTS search trees and experience replay buffers.

5.3.1.2 Job Management and Scalability

Jobs were orchestrated using the Slurm Workload Manager. To maximize efficiency, the AlphaZero implementation was configured to run in an actor-learner architecture:

- Actors: Multiple CPU-bound processes generating self-play games simultaneously.
- Learner: A GPU-bound process that pulls game trajectories from a shared queue to update the policy (π) and value (v) networks.
- Storage: A high-speed Lustre parallel file system was used to store model checkpoints and training logs, ensuring minimal I/O bottlenecks during the data-intensive training phases.

5.4 Heuristics

The heuristics suggested here are based on the idea, that the prior move probabilities express an inherent kind of certainty that can be leveraged to adjust the amount of simulation needed to generate a successful training sequence while lowering the computational efforts.

5.4.1 Difference of Probability for Likeliest K Actions

The Policy Confidence Margin heuristic adjusts the MCTS simulation budget based on the numerical "closeness" of the probabilities in the Policy Head output p .

5.4.1.1 The Mathematical Logic

We evaluate the gap between the top k probabilities. The most common approach is to look at the Top-2 Margin (ΔP):

$$\Delta P = P(s, a_{\text{best}}) - P(s, a_{\text{second}})$$

Low ΔP (Small Margin): The NN is uncertain. Two or more moves have nearly identical probabilities (e.g., 34% vs 32%). This is a “contested” state.

High ΔP (Large Margin): The NN is confident. One move dominates the distribution (e.g., 85% vs 5%). This is a “clear” state.

5.4.1.2 Reasoning: Strategic Bifurcation

On a 9x9 board, a small margin often occurs during a bifurcation in the game tree—a point where the game could swing wildly depending on a single choice.

Tie-Breaking: If the NN thinks two moves are equally good, it lacks the tactical depth to see the 10-move-deep consequence that makes one slightly better. Increasing simulations here allows the MCTS to “break the tie” effectively.

Resource Conservation: If the NN is 90% sure about a move, running 1,600 simulations usually just confirms what the “gut instinct” already knew. Reducing simulations here saves time for more complex positions.

5.4.1.3 Budget Scaling Function

You can implement a dynamic budget $N(s)$ using an inverse relationship to the margin:

$$N(s) = N_{\text{base}} \cdot (1 + \exp(-\tau \Delta P))$$

Where τ (temperature) is a hyperparameter that controls how aggressively you react to small margins.

Result: As $\Delta P \rightarrow 0$ (maximum uncertainty), the budget $N(s)$ doubles. As $\Delta P \rightarrow 1$ (maximum certainty), the budget stays near the base level.

5.4.2 Difference of Highest and Lowest K Actions

The Top-Bottom Policy Contrast heuristic adjusts the MCTS simulation budget by calculating the delta between the strongest and weakest move probabilities. This measures the “signal-to-noise ratio” of the current board position.

5.4.2.1 The Mathematical Logic

Given the sorted Policy vector $p = [p_1, p_2, \dots, p_n]$ where p_1 is the highest probability:

Top-5 Mean (M_{top}): The average probability of the five most likely moves.

$$M_{\text{top}} = \frac{1}{5} \sum_{i=1}^5 p_i$$

Bottom-5 Mean (M_{bot}): The average probability of the five least likely legal moves.

$$M_{\text{bot}} = \frac{1}{5} \sum_{i=n-4}^n p_i$$

The Contrast Delta (ΔC):

$$\Delta C = M_{\text{top}} - M_{\text{bot}}$$

5.4.2.2 Reasoning: Constrained vs. Unconstrained States

On the 9x9 board, this delta provides a unique “complexity signature” for the state:

- High Contrast ($\Delta C \uparrow$): The NN clearly distinguishes between high-value territory and “blunder” moves. This occurs in Life-and-Death (Tsumego) or late-game Yose.
 - Action: Reduce N. The NN has already “solved” the filtering; the search only needs to confirm the highest peak.
- Low Contrast ($\Delta C \downarrow$): The difference between the “best” and “worst” moves is negligible. This occurs in Opening (Fuseki) or highly complex Ko fights.
 - Action: Increase N. When the NN cannot differentiate between its top picks and its bottom picks, it is “blind” to the board’s features. A deep MCTS search is required to find the hidden structure.

5.4.3 Learning Complexity

The proposed algorithm adds a third “head” to the AlphaZero architecture or uses a post-hoc regression model to predict the Search Uncertainty (U) of a given state.

5.4.3.1 The Adaptive Simulation Heuristic

Instead of a fixed budget N, the number of simulations for a state s is determined by a learned function:

$$N(s) = N_{\min} + \Phi(s) \cdot (N_{\max} - N_{\min})$$

Where $\Phi(s) \in [0, 1]$ is the output of a Regression Head that estimates how much the MCTS “opinion” is likely to change if given more time.

5.4.3.2 Training the Regression Head

To train this model, you perform “High-Simulation” ground truth generation during the self-play phase:

- Search: Run a full MCTS with a very high budget (e.g., N=3200).
- Labeling: Record the “stability” of the move choice. If the best move at n=100 is the same as the best move at n=3200, the state has low uncertainty. If the preferred move flips multiple times during the search, the state has high uncertainty.
- Regression Loss: The NN is trained to minimize the Mean Squared Error (MSE) between its prediction $\Phi(s)$ and the measured stability of the search:

$$L_{\text{search}} = \|\Phi(s) - \text{Stability}(s)\|^2$$

5.4.3.3 Heuristics Leveraging NN Output

The regression model typically uses the existing Policy (p) and Value (v) outputs as its primary features:

- **Policy Entropy:** If the NN’s policy $P(s,a)$ is “flat” (e.g., five moves all have $\approx 20\%$ probability), the regression model should output high uncertainty, triggering more simulations to break the tie.
- **Value Contradiction:** If the Value Head v predicts a certain win, but the MCTS simulations start returning losses, this “surprise” or “misalignment” triggers an immediate increase in simulations to resolve the conflict.
- **9x9 Tactical Density:** On a 9x9 board, positions where stones are in close contact (high density of “Atari” or “Peeps”) are regressed toward higher simulation counts compared to empty-board openings.

5.4.4 Filtering Heuristic

We implement a Top-K or Soft-Threshold pruning mechanism. Before a node is expanded in the MCTS, the Policy Head (p) of the NN evaluates all legal moves $a \in A$. A move is only added to the search tree if it satisfies:

$$A_{\text{filtered}} = \{a \in A \mid P(s, a) > \sigma\}$$

Where σ is the pruning threshold (e.g., 0.01). Alternatively, a Top-K approach can be used where only the top 5, 10, or 15 moves are considered, regardless of their absolute probability.

Experimental Variables

- **Independent Variable:** The pruning threshold σ (tested at 0.0, 0.001, 0.01, and 0.05).
- **Dependent Variables:**
 - **Search Depth:** The maximum and average depth reached within a fixed time budget.
 - **Tactical Accuracy:** The AI’s ability to solve known “Life and Death” (Tsumego) problems.
 - **Win Rate:** Performance against a non-filtering baseline.

To quantify the efficiency gain, we conduct the following tests:

- **Fixed-Time Strength Test:** Both the Baseline (no filtering) and the Filtered AI are given 100ms per move. We measure how many additional plies (levels) of the tree the Filtered AI can explore due to the reduced branching factor.
- **Blunder Sensitivity Analysis:** We track how often the “optimal” move (as determined by a high-simulation 19x19 professional engine adapted for 9x9) is accidentally pruned by the σ threshold. This identifies the “Information Loss” of the heuristic.

5.4.5 Random

While the regression-based approach seeks the optimal budget, many state-of-the-art implementations utilize a Stochastic Simulation Heuristic. This involves randomly varying the number of MCTS playouts for different moves during self-play.

5.4.5.1 The “Fast vs. Slow” Search Heuristic

A common implementation involves a dual-budget strategy where the AI alternates between two modes:

- Full Search: The AI executes the maximum simulation budget (e.g., $N=800$) to find the most accurate move.
- Fast Search: The AI executes a minimal simulation budget (e.g., $N=100$) for a percentage of moves.

The Reasoning: By randomly selecting a lower simulation count for 75%–90% of moves during training, the AI can play significantly more games in the same amount of clock time. This exposes the Neural Network to a much wider variety of board states, which is often more beneficial for learning than seeing fewer games played with “perfect” accuracy.

5.4.5.2 Reasoning for Randomization in 9x9 Go

On a 9x9 board, the game is short (usually 30–60 moves). Randomizing the simulation budget serves three specific purposes:

- Noise Injection (Dirichlet-style): Similar to adding Dirichlet noise to the root node probabilities, randomizing the depth of the search prevents the agent from falling into “local optima” or repetitive opening patterns.
- Regularization: If the AI always uses 1600 simulations, the Policy Head (P) learns to rely on the MCTS “cleaning up” its mistakes. By forcing the AI to occasionally play with only 10 simulations, the Policy Head is forced to become more robust and accurate on its own (“gut instinct”).
- Sample Diversity: On 9x9, the branching factor is small. If simulations are always high, the AI might only ever see “perfect” play. Randomizing the budget simulates “human-like” variance, teaching the AI how to punish the suboptimal moves it might encounter in real-world play.

5.5 Evaluation

5.5.1 Self-Play Tournaments and Round-Robin Analysis

After the training phase, models are evaluated through a controlled tournament. Each model configuration (e.g., Vanilla UCT vs. PUCT vs. Policy Filtered) plays a series of games to generate a robust dataset for ranking.

- Match Volume: Each pair of agents competes in 400 games. This high volume is necessary to minimize the variance inherent in 9x9 Go, where a single tactical slip can decide the outcome.
- Komi Adjustment: To account for the first-move advantage, agents alternate as Black and White. We utilize a 7.5 point Komi under Chinese rules to ensure that games do not end in draws, providing a clear binary outcome for every match.

5.5.2 Strength Ranking: The BayesElo System

To transform tournament results into a global hierarchy, we utilize the BayesElo algorithm. Unlike a standard win percentage, BayesElo provides a standardized rating scale where the probability of a win is a function of the rating difference between two agents.

- **Likelihood Estimation:** The algorithm uses a Bayesian approach to find the most likely rating for each model given the observed game outcomes.
- **Uncertainty Quantification:** We calculate 95% Confidence Intervals (CI) for each rating. If the intervals of two heuristic configurations do not overlap, we can conclude with statistical certainty that one heuristic is superior to the other under the given simulation budget.

5.5.3 Efficiency Metrics: Elo-per-Simulation

A primary goal of this thesis is to reduce the computational cost of high-level play. Therefore, we do not just evaluate “who is stronger,” but “who is more efficient.”

- **Strength vs. Budget:** We test models at varying simulation counts ($n=100,400,1600,3200$). We plot the resulting BayesElo on a log-scale x-axis.
- **Heuristic Yield:** We measure the “Elo-per-Simulation” by calculating the slope of the strength curve. A successful heuristic—such as Policy Filtering—is expected to achieve a higher Elo at 400 simulations than the baseline achieves at 1600, representing a 4× increase in efficiency.

6 Experiments & Results

6.1 Using Heuristics

6.2 Using Heuristics in Play

7 Discussion

8 Conclusion

Bibliography